# Cahier des Charges — Audit critique & Restructuration

## Plateforme intelligente d'analyse, de classification et d'assistance sur tickets informatiques

> **Nature du document.** Audit de comité d'experts + cahier des charges restructuré, de niveau ingénieur / industriel. Rédigé par un comité fictif : professeur IA, encadrant PFE, ML Engineer senior, AI Architect, MLOps Engineer, expert RAG, expert Fine-Tuning LLM, Software Architect, expert en rédaction de cahiers des charges universitaires.
>
> **Principe directeur du comité :** un cahier des charges n'est **pas** un suivi d'avancement. Il définit *ce qui doit être construit*, *pourquoi*, *sous quelles exigences vérifiables*, et *comment on prouvera que c'est atteint*. Un pourcentage (« 85 % done ») n'a aucune valeur contractuelle ni scientifique : il ne dit ni le périmètre, ni les critères d'acceptation, ni les métriques cibles.
>
> **Principe d'honnêteté du comité :** nous distinguons explicitement ce qui est *indispensable*, ce qui est *différenciant à fort retour*, et ce qui serait *du sur-dimensionnement* (over- engineering) pour un PFE. Empiler les buzzwords (Kubernetes, multi-agents, knowledge graph…) sans justification est précisément le défaut qui fait perdre des points devant un jury exigeant.

---

# PARTIE I — AUDIT CRITIQUE DE L'EXISTANT

## I.0 Diagnostic transversal (le problème de fond)

Les éléments fournis (« ML/RAG pipeline — essentially done », « Backend FastAPI — 85 % done », « Frontend », « Deployment ») ne sont **pas des sections de cahier des charges** : ce sont des items d'un tableau d'avancement (*kanban*). Le jury a raison. Quatre défauts structurels :

1. **Confusion statut / spécification.** « done / 85 % » décrit un *état*, pas une *exigence*. Un cahier des charges écrit : *exigence → critère d'acceptation → métrique → moyen de vérification*. Exemple attendu : « EX-NF-03 : la latence P95 d'une prédiction de catégorie doit être ≤ 300 ms sous 50 req/s ; vérifiée par test de charge Locust ».
2. **Absence de problématique scientifique.** Aucune question de recherche n'est posée. Un PFE d'ingénieur IA doit formuler une *problématique* (ex. « comment classer des tickets IT francophones courts, déséquilibrés et bruités, avec une couverture fiable de la classe minoritaire *Sécurité* ? ») et une *hypothèse* falsifiable.
3. **Absence d'état de l'art et de justification des choix.** Le projet utilise TF-IDF + LogisticRegression (technologie de référence ~2003-2010) et un RAG TF-IDF/cosinus *sans embeddings ni base vectorielle*. Rien ne justifie ces choix par rapport aux alternatives modernes (CamemBERT, embeddings denses, LLM). Pour un diplôme d'ingénieur en 2026, l'absence de cette justification comparative est rédhibitoire.
4. **Aucune exigence non-fonctionnelle (ENF).** Pas de cible de performance, de sécurité, de scalabilité, d'observabilité, de gouvernance RGPD. Or les tickets contiennent des **données personnelles** (demandeur, filiale, descriptions) : un volet RGPD est *obligatoire*.

> **Constat du comité.** Le projet a une **bonne ingénierie sous-jacente** (anti-fuite de

> données par split sur textes uniques, routage par seuil de confiance, journal d'audit `TicketHistory`), mais il est **décrit comme un projet de développement logiciel, pas comme un projet d'IA**. La restructuration ci-dessous transforme ce suivi de tâches en cahier des charges scientifique.

---

## I.1 Audit de la section « ML / RAG pipeline — essentially done »

### 1.1.1 Pourquoi c'est insuffisant

- **« essentially done » n'est pas une exigence.** Aucun objectif chiffré, aucun protocole d'évaluation, aucun critère d'acceptation.
- **Modèle sous-dimensionné scientifiquement.** TF-IDF + LogReg est un *baseline* honnête, pas une *contribution*. Le jury attend au minimum une comparaison baseline → modèle profond (transformer francophone) → éventuellement LLM, avec benchmark chiffré.
- **Sur-apprentissage masqué.** `f1_macro_cv = 1.0` contre `f1_macro_test = 0.77`, sur un dataset **synthétique** (125 textes uniques sur 1800). Ce point doit être *assumé et traité* dans le cahier des charges (qualité des données, risque de non-généralisation), pas caché.
- **RAG archaïque.** `src/rag/engine.py` fait du TF-IDF + cosinus : **recherche lexicale**, pas **sémantique**. Il ne capte pas la synonymie (« impossible d'accéder à la base » ≠ « base indisponible »). Pas d'embeddings, pas de base vectorielle, pas de re-ranking, pas de LLM de génération, **aucune métrique RAG** (Precision@k, MRR, faithfulness…). Présenter cela comme « RAG » devant un jury IA est risqué : c'est un moteur de recommandation lexical.
- **Classe *Sécurité* à rappel ~20 %** (cf. `model_metadata.json`) : non traitée autrement que par des heuristiques mots-clés. Aucune stratégie scientifique (augmentation de données, focal loss, modèle dédié) n'est spécifiée.

### 1.1.2 Ce qui manque (scientifique / technique / industriel)

| Axe | Manque identifié |
|---|---|
| Scientifique | Problématique, hypothèses, état de l'art NLP francophone, protocole d'évaluation (validation croisée stratifiée, intervalle de confiance), traitement du déséquilibre, analyse d'erreurs, étude d'ablation. |
| Technique | Embeddings (modèles francophones), base vectorielle, hybrid search (BM25 + dense), re-ranking (cross-encoder), génération par LLM, gestion du contexte, guardrails anti-hallucination. |
| Industriel | Versionnage des données (DVC déjà présent mais `dvc.yaml` vide), versionnage des modèles (MLflow Registry), reproductibilité (`dvc repro`), monitoring de drift, optimisation du coût d'inférence. |

### 1.1.3 Nouvelle version (spécification cible)

Découper « ML/RAG » en **deux sous-systèmes spécifiés indépendamment** :

**A. Sous-système NLP de compréhension du ticket** — exigences :

- EX-IA-01 : classification multi-classe (8 catégories) — cible **F1-macro ≥ 0.85** sur jeu de test *réel/dé-dupliqué*, baseline TF-IDF+LogReg documentée comme référence basse.
- EX-IA-02 : modèle francophone de référence **CamemBERT**/**CamemBERTa** fine-tuné, comparé au baseline (benchmark obligatoire).
- EX-IA-03 : rappel classe *Sécurité* ≥ 0.70 (exigence dédiée car classe critique).
- EX-IA-04 : tâches NLP additionnelles spécifiées : **résumé automatique** du ticket, **extraction d'entités** (équipement, logiciel, code erreur, IP), **analyse de priorité**, **analyse de sentiment/urgence**.
- EX-IA-05 : explicabilité (SHAP/LIME pour la classification ; mise en évidence des tokens).
- EX-IA-06 : human-in-the-loop par seuil de confiance calibré (calibration de Platt / température), pas un

seuil magique 0.40 non justifié.

**B. Sous-système RAG d'assistance à la résolution** — exigences : voir chapitre dédié (Partie III, Ch. 7). En résumé : embeddings denses francophones + base vectorielle + hybrid search + re-ranking

- génération LLM avec citations + évaluation RAGAS.

### 1.1.4 Sous-sections à ajouter

1. Problématique et hypothèses de recherche.
2. État de l'art (NLP court francophone, RAG, LLM helpdesk).
3. Jeu de données : provenance, schéma, qualité, déséquilibre, **caractère synthétique assumé**, stratégie d'acquisition de données réelles.
4. Protocole d'évaluation et métriques.
5. Baselines et modèles avancés (benchmark).
6. Traitement du déséquilibre et de la classe minoritaire.
7. Explicabilité (XAI).
8. Calibration de la confiance et stratégie human-in-the-loop.

### 1.1.5 Pourquoi c'est important dans un PFE d'IA

Un PFE d'ingénieur est évalué sur la **démarche scientifique** (problème → état de l'art → méthode → expérimentation → analyse critique), pas sur le fait qu'un script « tourne ». Le benchmark baseline vs transformer est *la* preuve de maturité attendue.

---

## I.2 Audit de la section « Backend (FastAPI) — 85 % done »

### 1.2.1 Pourquoi c'est insuffisant

- **Le « 85 % » ne réfère à aucun périmètre défini.** 85 % de quoi ? Sans liste d'exigences, c'est invérifiable.
- **Aucune exigence non-fonctionnelle** : latence, débit, disponibilité, sécurité, observabilité.
- **Couches vides** : `services/`, `db/repositories/`, `middleware/` sont des squelettes ; la logique est dans les routers. À spécifier comme dette d'architecture (séparation des responsabilités) ou à assumer comme choix « monolithe modulaire ».
- **Endpoint `/ingest` cassé** (import `datetime` manquant, mauvais appel) et non enregistré : un cahier des charges doit définir le *contrat d'API* et les critères d'acceptation par endpoint.

### 1.2.2 Ce qui manque

- **Contrat d'API** (OpenAPI) versionné, codes d'erreur, schémas requête/réponse, pagination, idempotence des endpoints d'ingestion.
- **ENF** : P95 latence, débit cible, taux d'erreur, RTO/RPO.
- **Sécurité applicative** : RBAC complet, gestion des secrets (le `SECRET_KEY=change-me` en clair dans `.env` est une faille à corriger et à spécifier), rate limiting, validation/sanitization, protection contre l'injection de prompt côté LLM.
- **Architecture** : choix monolithe modulaire vs microservices *justifié* (pas imposé).

### 1.2.3 Nouvelle version (spécification cible)

- EX-API-01 : contrat OpenAPI 3.1 complet, versionné (`/api/v1`).
- EX-NF-01 : latence P95 prédiction ≤ 300 ms ; RAG (avec LLM) P95 ≤ 3 s.
- EX-NF-02 : disponibilité cible 99 % en démonstration ; *graceful degradation* si le LLM est indisponible (repli sur recherche sans génération).
- EX-SEC-01 : authentification JWT + RBAC (client/agent/admin), secrets via *secret manager* / variables

d'environnement chiffrées, rotation des clés.
- EX-SEC-02 : protection contre l'injection de prompt et la fuite de données via le LLM (guardrails entrée/sortie).

### 1.2.4 Sous-sections à ajouter

Contrat d'API ; exigences non-fonctionnelles ; sécurité & menaces (modèle STRIDE léger) ; architecture en couches / modularité ; stratégie de tests (unitaires, intégration, contractuels).

### 1.2.5 Pourquoi c'est important

L'aspect « industriel » d'un PFE se juge sur les **ENF et la sécurité**, pas sur le nombre d'endpoints. Un backend « 85 % » sans cible de latence ni modèle de menace n'est pas défendable.

---

## I.3 Audit de la section « Frontend »

### 1.3.1 Pourquoi c'est insuffisant

Un seul mot. Aucune exigence d'expérience utilisateur, d'accessibilité, ni de spécification des écrans. État réel constaté : Angular 21 ; mais `environment.ts` **vide** (URL API `undefined`), *enum* de statuts **désaligné** du backend, modèle `Ticket` **sans les champs IA** (`confiance`, `categorie_finale`, `solution_rag`), page de détail **manquante**, **aucun guard** sur `/admin`. Donc : le front *ne communique pas encore avec le back*.

### 1.3.2 Ce qui manque

Spécification des parcours (client / agent / admin), maquettes/wireframes, exigences UX (restitution de la confiance et des sources RAG = enjeu d'**IA explicable côté UI**), gestion des états de chargement/erreur, accessibilité (RGAA/WCAG), responsive, sécurité front (guards, refresh token), internationalisation.

### 1.3.3 Nouvelle version (spécification cible)

- EX-UX-01 : 3 personas et leurs parcours spécifiés (use cases UML).
- EX-UX-02 : l'écran agent affiche **catégorie prédite + score de confiance + top-k suggestions RAG avec citations cliquables vers le ticket source** (explicabilité utilisateur).
- EX-UX-03 : feedback utilisateur capturé (👍/👎 + correction de catégorie) → boucle d'amélioration (active learning).
- EX-UX-04 : guards d'authentification/rôles, gestion centralisée des erreurs (intercepteur HTTP).

### 1.3.4 Sous-sections à ajouter

Personas & parcours ; maquettes ; exigences UX/accessibilité ; intégration contrat d'API ; restitution de l'explicabilité ; collecte de feedback ; sécurité front.

### 1.3.5 Pourquoi c'est important

L'interface est le lieu du **human-in-the-loop** : c'est là que la confiance, les sources et le feedback se matérialisent. C'est un argument scientifique (boucle de rétroaction), pas cosmétique.

---

## I.4 Audit de la section « Deployment »

### 1.4.1 Pourquoi c'est insuffisant

Un mot, aucune cible. Pas d'environnement défini, pas de CI/CD, pas d'IaC, pas d'observabilité, pas de stratégie

de scalabilité ni de coûts.

### 1.4.2 Ce qui manque

Conteneurisation multi-services (API, front, base vectorielle, LLM/serving), orchestration (docker-compose suffisant pour un PFE ; **Kubernetes = sur-dimensionnement** sauf si la scalabilité est un objectif explicite), CI/CD (GitHub Actions), registre d'images, monitoring/observabilité (Prometheus/Grafana, logs structurés, tracing), gestion des secrets, stratégie cloud, plan de *rollback*, **optimisation du coût d'inférence LLM** (quantization, cache, batching).

### 1.4.3 Nouvelle version (spécification cible)

- EX-OPS-01 : chaque service conteneurisé ; `docker-compose` reproductible en une commande.
- EX-OPS-02 : pipeline CI/CD (lint, tests, build, scan de sécurité d'image, déploiement).
- EX-OPS-03 : observabilité : métriques techniques (latence, erreurs) **et** métriques IA (distribution des confiances, taux de fallback RAG, taux de hallucination estimé, drift).
- EX-OPS-04 : détection de drift des données/prédictions + alerting.
- EX-OPS-05 : maîtrise du coût d'inférence (mesure €/1000 tickets ; stratégies de réduction).

### 1.4.4 Sous-sections à ajouter

Topologie de déploiement ; CI/CD ; IaC ; observabilité technique & IA ; drift & ré-entraînement ; coûts d'inférence ; plan de reprise.

### 1.4.5 Pourquoi c'est important

Le **MLOps/LLMOps** différencie un projet d'IA « jouet » d'un projet « industriel ». Le monitoring *spécifique IA* (drift, hallucination, coût) est attendu d'un ingénieur, pas seulement le monitoring CPU/RAM.

---

# PARTIE II — COMPOSANTS D'UN PROJET D'IA MODERNE : MATRICE DE PERTINENCE

> Le comité refuse le « tout cocher ». Voici une matrice honnête **adaptée à ce projet précis** (classification + assistance sur tickets IT francophones). Priorités : **M** = Must (indispensable au niveau ingénieur), **S** = Should (fort différenciateur, recommandé), **C** = Could (bonus si temps), **O** = Over-engineering pour un PFE solo (à citer en perspective seulement).

| Composant | Pertinence | Priorité | Comment l'intégrer ici |
|---|---|---|---|
| Machine Learning classique | Oui | **M** | Baseline TF-IDF+LogReg/SVM comme référence de comparaison. |
| Deep Learning (Transformers) | Oui | **M** | **CamemBERT/CamemBERTa** fine-tuné = cœur de la montée en gamme NLP. |
| Fine-Tuning (LoRA/QLoRA) | Oui | **S** | LoRA sur CamemBERT (classif) et/ou QLoRA sur petit LLM pour génération de réponses. Différenciateur fort. |
| RAG | Oui | **M** | Refondre en RAG sémantique (embeddings + vector DB + génération). |
| Agents IA | Partiellement | **C** | Un *agent orchestrateur* (classif → RAG → réponse → escalade) est défendable. |

| | | | |
|---|---|---|---|
| Multi-Agent System | Limité | **O** | Sur-dimensionné pour ce périmètre ; citer en perspective. |
| MLOps | Oui | **M** | DVC (déjà là, à activer) + MLflow Registry + pipeline reproductible. |
| LLMOps | Oui | **S** | Prompt versioning, éval automatique, tracing des appels LLM, coûts. |
| Prompt Engineering | Oui | **M** | Templates versionnés, few-shot, garde-fous, format de sortie contraint. |
| Vector Database | Oui | **M** | Qdrant ou Chroma (ou `pgvector` si Postgres). FAISS si embarqué. |
| Knowledge Base | Oui | **M** | Structurer les tickets résolus + docs ; chunking + métadonnées. |
| FastAPI | Oui | **M** | Déjà en place ; à durcir (contrat, sécurité). |
| Docker | Oui | **M** | Déjà prévu ; multi-services. |
| Kubernetes | Non (sauf objectif scalabilité) | **O** | `docker-compose` suffit ; K8s = perspective. |
| CI/CD | Oui | **S** | GitHub Actions : lint, tests, build, scan, déploiement. |
| Monitoring / Observabilité | Oui | **S** | Prometheus/Grafana + logs structurés + tracing ; **+ métriques IA**. |
| Sécurité | Oui | **M** | RBAC, secrets, injection de prompt, durcissement. |
| Gouvernance des données | Oui | **M** | **RGPD obligatoire** (données personnelles), anonymisation, rétention. |
| Évaluation des modèles | Oui | **M** | Protocole rigoureux, IC, analyse d'erreurs, ablation. |
| Benchmark | Oui | **M** | Baseline vs transformer vs LLM, tableaux comparatifs. |
| Explainable AI | Oui | **S** | SHAP/LIME (classif) + attributions/citations (RAG). |
| Human-in-the-loop | Oui | **M** | Déjà amorcé (file de validation) ; à formaliser + feedback loop. |
| Optim. coût d'inférence | Oui | **S** | Quantization, cache sémantique, batching, choix taille modèle. |
| Scalabilité | Oui | **S** | Asynchrone, batching embeddings, stateless API. |
| Microservices | Optionnel | **C/O** | Monolithe modulaire bien découpé = défendable ; microservices = perspective. |

# PARTIE III — STRUCTURE COMPLÈTE DU CAHIER DES CHARGES

> Pour **chaque chapitre** : *Objectif · Contenu attendu · Diagrammes · Tableaux · Métriques · Risques · Livrables.*

## Chapitre 1 — Introduction générale

- **Objectif** : poser le contexte, la problématique scientifique, les objectifs et les contributions.
- **Contenu** : contexte métier (helpdesk IT, volumétrie, coût d'un ticket : 30–45 min → cible 5–10 min), **problématique** (classification de tickets courts francophones déséquilibrés + assistance fiable à la résolution), **questions de recherche**, **hypothèses**, objectifs SMART, contributions, périmètre (in/out of

scope).
- **Diagrammes** : diagramme de contexte (système et acteurs).
- **Tableaux** : objectifs SMART ; périmètre inclus/exclu.
- **Métriques** : KPIs métier cibles (temps de traitement, taux d'auto-routage fiable, satisfaction).
- **Risques** : périmètre trop large ; données synthétiques.
- **Livrables** : note de cadrage, problématique validée.

## Chapitre 2 — Étude de l'existant et état de l'art

- **Objectif** : justifier scientifiquement les choix par comparaison à l'existant.
- **Contenu** : solutions du marché (Zendesk/Freshdesk/ServiceNow + IA) ; état de l'art NLP texte court ; classification déséquilibrée ; **NLP francophone (CamemBERT, FlauBERT, CamemBERTa)** ; RAG (dense retrieval, hybrid search, re-ranking) ; LLM et fine-tuning (LoRA/QLoRA) ; évaluation RAG (RAGAS) ; positionnement du projet.
- **Diagrammes** : carte de positionnement (axes : sémantique vs lexical, coût vs qualité).
- **Tableaux** : comparatif solutions ; comparatif modèles candidats (taille, langue, licence, coût, perf).
- **Métriques** : critères de sélection pondérés.
- **Risques** : choix non justifiés ; licences (modèles/commerciaux).
- **Livrables** : revue de littérature, grille de choix technologiques.

## Chapitre 3 — Analyse et spécification des besoins

- **Objectif** : exigences fonctionnelles (EF) et non-fonctionnelles (ENF) vérifiables.
- **Contenu** : personas (client, agent/technicien, admin/responsable) ; user stories ; priorisation **MoSCoW** ; EF (création, classification, résumé, extraction d'entités, priorité, sentiment, suggestion RAG, validation, historique, feedback) ; **ENF** (perf, sécurité, fiabilité, maintenabilité, RGPD, accessibilité) ; règles de gestion ; matrice de traçabilité exigences ↔ tests.
- **Diagrammes UML** : cas d'utilisation ; **diagramme d'activité** du workflow ticket ; **diagramme de séquence** (création → prédiction → RAG → validation) ; diagramme d'états du statut de ticket.
- **Tableaux** : EF (ID, description, priorité MoSCoW, critère d'acceptation) ; ENF (ID, métrique, cible, vérification) ; matrice de traçabilité.
- **Métriques** : cibles ENF (P95, dispo, etc.).
- **Risques** : exigences ambiguës ; *scope creep*.
- **Livrables** : SRS (Software Requirements Specification).

## Chapitre 4 — Conception et architecture

- **Objectif** : définir l'architecture logique, applicative, données et de déploiement.
- **Contenu** : architecture en couches / modulaire (justifier monolithe modulaire vs microservices) ; vue logique des sous-systèmes (NLP, RAG, API, UI, données, MLOps) ; choix technologiques justifiés ; **ADR (Architecture Decision Records)**.
- **Diagrammes** : architecture **C4** (Contexte, Conteneurs, Composants) ; diagramme de déploiement ; **diagramme de flux de données** (du ticket à la réponse) ; diagramme de classes (domaine : User, Ticket, TicketHistory, Feedback, KBDocument).
- **Tableaux** : décisions d'architecture (ADR) ; stack par couche.
- **Métriques** : couplage/cohésion (qualitatif), points de scalabilité.
- **Risques** : sur-architecture ; dette technique.
- **Livrables** : dossier de conception + ADRs.

## Chapitre 5 — Données et gouvernance

- **Objectif** : spécifier données, qualité, RGPD, versionnage.

- **Contenu** : schéma des données (23 colonnes du CSV), **audit qualité** (doublons, NaN, déséquilibre 5,9×, **caractère synthétique : 125 textes uniques/1800**), stratégie d'acquisition de données réelles, **anonymisation/pseudonymisation (RGPD)**, politique de rétention, versionnage **DVC** (activer `dvc.yaml` : preprocess → split → train → eval), lignage des données.
- **Diagrammes** : pipeline de données (DVC DAG).
- **Tableaux** : dictionnaire de données ; rapport qualité ; registre RGPD (donnée, finalité, base légale, durée).
- **Métriques** : % complétude, % doublons, ratio de déséquilibre, taux de PII anonymisées.
- **Risques** : non-conformité RGPD ; non-généralisation (synthétique).
- **Livrables** : data card, pipeline DVC reproductible, registre de traitement.

## Chapitre 6 — Modélisation IA (NLP)

- **Objectif** : spécifier et évaluer les modèles de compréhension du ticket.
- **Contenu** : prétraitement ; **baseline** (TF-IDF + LogReg/SVM) ; **modèle profond** (CamemBERT/ CamemBERTa fine-tuné, LoRA) ; tâches : classification, **résumé** (modèle séquence-à- séquence type Barthez/mT5 ou LLM), **NER** (équipement, logiciel, code erreur, IP), **priorité**, **sentiment/urgence** ; traitement du déséquilibre (pondération, augmentation, focal loss) ; **calibration** de la confiance ; **XAI** (SHAP/LIME) ; étude d'ablation.
- **Diagrammes** : pipeline d'entraînement ; matrice de confusion ; courbes (PR, calibration).
- **Tableaux** : **benchmark** (modèle × métriques) ; hyperparamètres ; analyse d'erreurs par classe ; ablation.
- **Métriques** : F1-macro/micro, précision/rappel par classe (**focus Sécurité**), MCC, ECE (calibration), temps d'inférence.
- **Risques** : sur-apprentissage (déjà visible), coût GPU, données insuffisantes.
- **Livrables** : modèles versionnés (MLflow), model cards, rapport d'évaluation.

## Chapitre 7 — Système RAG d'assistance

- **Objectif** : spécifier un RAG **sémantique** moderne, évalué.
- **Contenu** : ingestion & **chunking** de la base de connaissance (tickets résolus + procédures) ; **embeddings francophones** (ex. `intfloat/multilingual-e5`, `Solon`, `Sentence-CamemBERT`) ; **base vectorielle** (Qdrant/Chroma/pgvector) ; **hybrid search (BM25 + dense)** ; **re-ranking** (cross-encoder) ; **génération** par LLM avec **citations** et format contraint ; **guardrails** & **détection d'hallucinations** ; cache sémantique ; stratégie de repli (sans LLM).
- **Diagrammes** : architecture RAG (indexation + requête) ; séquence requête→réponse.
- **Tableaux** : comparatif embeddings ; comparatif stratégies (lexical vs dense vs hybride vs +rerank) ; paramètres (chunk size, top-k).
- **Métriques** : **retrieval** : Precision@k, Recall@k, MRR, nDCG, Context Precision/Recall ; **génération** : Faithfulness, Answer Relevancy, Hallucination Rate (via **RAGAS** + **LLM-as-a-Judge**) ; latence ; coût/ requête.
- **Risques** : hallucinations, latence/coût LLM, fuite de données via le contexte.
- **Livrables** : index vectoriel versionné, harnais d'évaluation RAG, rapport comparatif.

## Chapitre 8 — LLM, Fine-Tuning & Prompt Engineering

- **Objectif** : spécifier l'usage du LLM, l'éventuel fine-tuning et la gestion des prompts.
- **Contenu** : choix du LLM (ouvert/local type Mistral/Llama francophone vs API) avec arbitrage coût/qualité/ confidentialité ; **fine-tuning LoRA/QLoRA** (jeu d'instructions construit à partir des résolutions historiques) ; **prompt engineering** (templates, few-shot, sortie structurée) ; **versionnage des prompts** ; **guardrails** (entrée : anti-injection ; sortie : format, PII, toxicité) ; politique de *fallback*.
- **Diagrammes** : pipeline de fine-tuning ; flux de garde-fous.
- **Tableaux** : datasets d'instruction ; hyperparamètres LoRA ; registre de prompts (version, diff, métrique associée) ; comparatif LLM (coût, latence, qualité, confidentialité).

- **Métriques** : qualité des réponses (LLM-as-a-Judge), taux d'hallucination, coût/1000 réponses, gain du fine-tuning vs zero-shot.
- **Risques** : confidentialité (envoi de données à une API externe), coût, dérive de prompt.
- **Livrables** : adaptateurs LoRA versionnés, bibliothèque de prompts, politique guardrails.

## Chapitre 9 — MLOps & LLMOps

- **Objectif** : industrialiser cycle de vie modèles et LLM.
- **Contenu** : pipeline reproductible (DVC) ; **registre de modèles** (MLflow) ; CI/CD modèles ; **monitoring de production** (technique + IA) ; **drift detection** (données & prédictions) ; ré-entraînement déclenché ; **observabilité LLM** (tracing des appels, tokens, coûts) ; **active learning** sur le feedback agent.
- **Diagrammes** : MLOps loop (données→train→registry→déploiement→monitoring→feedback).
- **Tableaux** : matrice d'observabilité (métrique, source, seuil, alerte) ; plan de ré- entraînement.
- **Métriques** : drift (PSI/KL), fraîcheur du modèle, MTTR, coût d'inférence, % feedback réinjecté.
- **Risques** : drift silencieux, dette MLOps, coûts non maîtrisés.
- **Livrables** : pipelines CI/CD, dashboards de monitoring, runbooks.

## Chapitre 10 — Développement applicatif (API & Front)

- **Objectif** : spécifier la réalisation logicielle.
- **Contenu** : contrat **OpenAPI** ; couches (routers/services/repositories) ; sécurité (JWT, RBAC, secrets, rate limiting, anti-injection de prompt) ; front Angular (parcours, restitution confiance + sources, feedback, guards, i18n, accessibilité) ; intégration front↔back (corriger `environment.ts`, aligner enum statuts et modèle `Ticket`).
- **Diagrammes** : séquence d'authentification ; composants front.
- **Tableaux** : endpoints (méthode, rôle, critère d'acceptation) ; plan de tests.
- **Métriques** : couverture de tests, conformité accessibilité, P95 endpoints.
- **Risques** : désalignement contrat front/back (déjà constaté), failles sécurité.
- **Livrables** : API documentée, front intégré, suite de tests.

## Chapitre 11 — Déploiement & Infrastructure

- **Objectif** : spécifier la mise en production.
- **Contenu** : conteneurisation multi-services ; `docker-compose` ; CI/CD ; observabilité ; gestion des secrets ; cible cloud ; scalabilité (asynchrone, batching) ; coûts.
- **Diagrammes** : diagramme de déploiement ; topologie réseau.
- **Tableaux** : services & ports ; budget ressources/coûts.
- **Métriques** : temps de build/déploiement, dispo, €/mois, €/1000 tickets.
- **Risques** : sur-dimensionnement (K8s), coûts cloud, secrets exposés.
- **Livrables** : infrastructure conteneurisée, pipeline CI/CD, doc d'exploitation.

## Chapitre 12 — Évaluation, validation & expérimentation

- **Objectif** : prouver l'atteinte des objectifs.
- **Contenu** : protocole expérimental ; jeux de test ; résultats NLP ; résultats RAG ; tests système (charge, sécurité) ; **étude utilisateur** (agents) ; analyse critique et limites.
- **Diagrammes** : courbes de résultats ; comparatifs.
- **Tableaux** : synthèse des métriques vs cibles (matrice exigence→résultat).
- **Métriques** : toutes les cibles des chapitres 6–9 + KPIs métier.
- **Risques** : sur-interprétation sur données synthétiques.
- **Livrables** : rapport d'évaluation, dashboards, démonstration.

### Chapitre 13 — Gestion de projet

- **Objectif** : démontrer la maîtrise méthodologique.
- **Contenu** : méthodologie (agile/itératif), planning (Gantt), jalons, **gestion des risques** (registre), organisation, outillage.
- **Diagrammes** : Gantt ; *burndown* (optionnel).
- **Tableaux** : **registre des risques** (risque, probabilité, impact, mitigation) ; RACI léger.
- **Métriques** : avancement par jalon, vélocité.
- **Risques** : voir registre.
- **Livrables** : planning, registre de risques, comptes-rendus.

### Chapitre 14 — Conclusion & perspectives

- **Objectif** : bilan critique et ouverture.
- **Contenu** : contributions, limites assumées (données synthétiques, etc.), perspectives (multi-agents, knowledge graph, K8s, continual learning) clairement positionnées comme *futures*.
- **Livrables** : synthèse, feuille de route.

---

# PARTIE IV — AXES D'INNOVATION (priorisés)

> Le différentiateur d'un PFE « excellent » n'est pas la quantité d'innovations, mais leur **pertinence justifiée et leur évaluation rigoureuse**. Priorité réaliste pour un PFE.

| # | Innovation | Apport pour ce projet | Effort | Priorité |
|---|---|---|---|---|
| 1 | **CamemBERT fine-tuné (LoRA)** pour la classification | Saut de qualité vs TF-IDF ; adapté au **français** | Moyen | **Must** |
| 2 | **RAG sémantique** (embeddings + vector DB) | Remplace le RAG lexical actuel ; capte la synonymie | Moyen | **Must** |
| 3 | **Évaluation RAG** (Precision@k, Recall@k, MRR, nDCG, Faithfulness, Answer Relevancy, Context Precision/Recall, Hallucination Rate) via **RAGAS** | Preuve scientifique du RAG ; rare dans les PFE → très valorisé | Moyen | **Must** |
| 4 | **Hybrid Search (BM25 + dense)** | Robustesse sur sigles/codes erreurs IT | Faible | **Should** |
| 5 | **Re-ranking** (cross-encoder) | Améliore la précision du top-k | Faible | **Should** |
| 6 | **LLM-as-a-Judge** | Évaluation automatique des réponses générées | Faible | **Should** |
| 7 | **Guardrails + détection d'hallucinations** | Sécurité/fiabilité des réponses LLM | Moyen | **Should** |
| 8 | **Génération de réponse par LLM avec citations** | Passe de « recommandation » à « assistance » réelle | Moyen | **Should** |
| 9 | **QLoRA d'un petit LLM** sur résolutions historiques | Modèle de réponse spécialisé, déployable localement | Élevé | **Could** |
| 10 | **Prompt versioning** | Traçabilité/reproductibilité LLMOps | Faible | **Should** |

| | | | | |
|---|---|---|---|---|
| 11 | **Drift detection + monitoring IA** | Volet production/MLOps | Moyen | **Should** |
| 12 | **Active learning** (feedback agent → ré-entraînement) | Boucle vertueuse human-in-the-loop | Moyen | **Could** |
| 13 | **Agent orchestrateur** (classif→RAG→réponse→escalade) | Architecture agentique pertinente | Moyen | **Could** |
| 14 | **Calibration de la confiance** (Platt/température) | Seuil de routage *fondé* et non arbitraire | Faible | **Should** |
| 15 | **XAI** (SHAP/LIME + attributions RAG) | Explicabilité exigée en IA responsable | Faible | **Should** |
| 16 | **Optimisation coût d'inférence** (quantization, cache sémantique) | Volet industriel/coûts | Moyen | **Could** |
| 17 | Knowledge Graph / Multi-Agent / Kubernetes / Continual Learning | Potentiel réel mais **hors périmètre PFE** | Élevé | **Perspective** |

# PARTIE V — VERSION FINALE : SOMMAIRE PROFESSIONNEL EXPLOITABLE

```
RÉSUMÉ / ABSTRACT (FR + EN)
REMERCIEMENTS · LISTE DES FIGURES · TABLEAUX · ACRONYMES

CHAPITRE 1 — INTRODUCTION GÉNÉRALE
  1.1 Contexte métier et organisationnel
  1.2 Problématique et questions de recherche
  1.3 Hypothèses
  1.4 Objectifs (SMART) et contributions
  1.5 Périmètre (in / out of scope)
  1.6 Organisation du document

CHAPITRE 2 — ÉTAT DE L'ART ET ÉTUDE DE L'EXISTANT
  2.1 Solutions de gestion de tickets et IA du marché
  2.2 NLP pour textes courts et classification déséquilibrée
  2.3 Modèles de langue francophones (CamemBERT, FlauBERT, CamemBERTa)
  2.4 RAG : dense retrieval, hybrid search, re-ranking
  2.5 LLM, fine-tuning (LoRA/QLoRA), prompt engineering
  2.6 Évaluation des systèmes RAG (RAGAS, LLM-as-a-Judge)
  2.7 Positionnement et grille de choix technologiques

CHAPITRE 3 — ANALYSE ET SPÉCIFICATION DES BESOINS
  3.1 Acteurs et personas
  3.2 Exigences fonctionnelles (MoSCoW)
  3.3 Exigences non-fonctionnelles (perf, sécurité, RGPD, accessibilité…)
  3.4 Règles de gestion et workflow
  3.5 Modélisation UML (cas d'utilisation, activité, séquence, états)
  3.6 Matrice de traçabilité exigences ↔ tests

CHAPITRE 4 — CONCEPTION ET ARCHITECTURE
  4.1 Architecture globale (C4 : contexte, conteneurs, composants)
  4.2 Sous-systèmes (NLP, RAG, API, UI, données, MLOps)
  4.3 Modèle de données (diagramme de classes)
  4.4 Choix technologiques et ADRs
```

```
  4.5 Vue de déploiement et flux de données

CHAPITRE 5 — DONNÉES ET GOUVERNANCE
  5.1 Sources et dictionnaire de données
  5.2 Audit qualité (doublons, NaN, déséquilibre, synthétique)
  5.3 Conformité RGPD et anonymisation
  5.4 Versionnage et lignage (DVC)
  5.5 Stratégie d'acquisition de données réelles

CHAPITRE 6 — MODÉLISATION IA (NLP)
  6.1 Prétraitement
  6.2 Baseline (TF-IDF + LogReg/SVM)
  6.3 Modèles profonds francophones (CamemBERT + LoRA)
  6.4 Tâches : classification, résumé, NER, priorité, sentiment
  6.5 Traitement du déséquilibre et de la classe Sécurité
  6.6 Calibration de la confiance et human-in-the-loop
  6.7 Explicabilité (SHAP/LIME)
  6.8 Benchmark, analyse d'erreurs, ablation

CHAPITRE 7 — SYSTÈME RAG
  7.1 Ingestion, chunking, base de connaissance
  7.2 Embeddings francophones
  7.3 Base vectorielle
  7.4 Hybrid search (BM25 + dense)
  7.5 Re-ranking
  7.6 Génération avec citations et guardrails
  7.7 Détection d'hallucinations
  7.8 Évaluation RAG (Precision@k, Recall@k, MRR, nDCG, Faithfulness,
      Answer Relevancy, Context Precision/Recall, Hallucination Rate)

CHAPITRE 8 — LLM, FINE-TUNING & PROMPT ENGINEERING
  8.1 Choix du LLM (arbitrage coût/qualité/confidentialité)
  8.2 Fine-tuning LoRA/QLoRA
  8.3 Prompt engineering et versionnage
  8.4 Guardrails (injection, PII, format, toxicité)

CHAPITRE 9 — MLOPS & LLMOPS
  9.1 Pipeline reproductible (DVC) et registre de modèles (MLflow)
  9.2 CI/CD des modèles
  9.3 Monitoring technique et IA
  9.4 Drift detection et ré-entraînement
  9.5 Observabilité LLM et coûts
  9.6 Active learning sur le feedback

CHAPITRE 10 — DÉVELOPPEMENT APPLICATIF
  10.1 Contrat d'API (OpenAPI) et architecture en couches
  10.2 Sécurité applicative (JWT, RBAC, secrets, anti-injection)
  10.3 Frontend (parcours, explicabilité UI, feedback, guards, i18n)
  10.4 Stratégie de tests

CHAPITRE 11 — DÉPLOIEMENT & INFRASTRUCTURE
  11.1 Conteneurisation et orchestration (docker-compose)
  11.2 CI/CD et IaC
  11.3 Observabilité et alerting
  11.4 Scalabilité et coûts

CHAPITRE 12 — ÉVALUATION & VALIDATION
  12.1 Protocole expérimental
  12.2 Résultats NLP / RAG / système
```

```
  12.3 Étude utilisateur
  12.4 Synthèse exigences → résultats et analyse critique

CHAPITRE 13 — GESTION DE PROJET
  13.1 Méthodologie et planning (Gantt)
  13.2 Registre des risques
  13.3 Organisation et outillage

CHAPITRE 14 — CONCLUSION & PERSPECTIVES
  14.1 Bilan et contributions
  14.2 Limites assumées
  14.3 Perspectives (multi-agents, KG, K8s, continual learning)

ANNEXES : data card, model cards, ADRs, contrat OpenAPI, registre de prompts,
          résultats détaillés, registre RGPD.
```

---

## Synthèse des livrables transverses attendus

- **Diagrammes** : contexte, C4 (×3), cas d'utilisation, activité (workflow), séquence (×2), états, classes, déploiement, flux de données, pipelines (données/MLOps/RAG/fine-tuning), Gantt.
- **Tableaux clés** : objectifs SMART, EF (MoSCoW), ENF, traçabilité, dictionnaire de données, rapport qualité, registre RGPD, ADRs, benchmark NLP, comparatif embeddings, comparatif stratégies RAG, registre de prompts, matrice d'observabilité, registre des risques, synthèse exigences→résultats.
- **Métriques clés** : F1-macro & par classe (focus Sécurité), MCC, ECE ; Precision@k, Recall@k, MRR, nDCG, Context Precision/Recall, Faithfulness, Answer Relevancy, Hallucination Rate ; latence P95, débit, dispo ; drift (PSI/KL) ; coût €/1000 tickets.
- **Risques majeurs** : données synthétiques (non-généralisation), hallucinations LLM, coût/latence d'inférence, conformité RGPD, sur-dimensionnement, sécurité (secrets, injection de prompt).

## Recommandation finale du comité (priorisation réaliste pour le PFE)

1. **Reformuler le projet en problématique scientifique** (Ch. 1–3) — *sans cela, rien ne sauve la note*.
2. **Monter en gamme le NLP** : CamemBERT fine-tuné + benchmark vs baseline (Ch. 6).
3. **Refondre le RAG en sémantique + l'évaluer avec RAGAS** (Ch. 7) — *plus gros différenciateur*.
4. **Ajouter le volet MLOps/LLMOps minimal** : DVC actif, MLflow, monitoring IA, prompt versioning (Ch. 9).
5. **Durcir API/sécurité + finaliser l'intégration front** (Ch. 10).
6. **Conteneuriser + CI/CD + observabilité** (Ch. 11).
7. **Garder en perspective** (et non en périmètre) : multi-agents, knowledge graph, Kubernetes, continual learning.